МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Государственный Университет Управления

Кафедра: Институт Информационных Систем

Лабораторная работа № 7

Рекурсивные алгоритмы и динамическое программирование в задачах комбинаторной оптимизации

Дисциплина: Теория алгоритмов

Направление подготовки: 38.03.05 «Бизнес-информатика»

Язык реализации: Python 3.10+

Вариант: 14

Выполнил: студент группы БИ 2-1 Лапшенков Григорий Витальевич

Проверил: [должность, ФИО преподавателя]

[Город], 2026

1. Цель работы и задание варианта

1.1. Цель работы

Изучить метод динамического программирования как способ устранения повторных вычислений в рекурсивных алгоритмах и экспериментально сопоставить трудоёмкость трёх реализаций одной задачи — наивной рекурсивной, рекурсивной с мемоизацией и итеративной.

1.2. Задание варианта 14

Таблица 1. Исходные данные варианта

Параметр	Значение
Задача А (три реализации)	числа Фибоначчи, параметр $n = 10 \dots 32$
Задача Б (динамическое программирование)	размен суммы: $S = 180$, номиналы 1, 5, 12
Дополнительное задание Г7	прикладная задача: отбор проектов в портфель при ограниченном бюджете; график зависимости прибыли портфеля от бюджета

Требуется: реализовать задачу А тремя способами со счётчиком вызовов; построить дерево рекурсии для малого n; измерить время и найти n, при котором наивная реализация перестаёт завершаться за 10 с; решить задачу Б методом ДП с восстановлением набора монет; сверить результат с полным перебором; решить задачу Г7.

2. Рекуррентные соотношения

2.1. Задача А: числа Фибоначчи

$$F(n) = F(n-1) + F(n-2), \qquad F(0) = 0,\ F(1) = 1.$$

- $F(0) = 0$ и $F(1) = 1$ — базовые случаи, на которых рекурсия прекращается;
- $F(n-1)$ и $F(n-2)$ — решения двух подзадач меньшего размера; их сумма — решение исходной задачи.

Число вызовов наивной версии $T(n)$ подчиняется рекуррентному соотношению

$$T(n) = T(n-1)+T(n-2)+1, \quad T(0) = T(1) = 1 \quad \Rightarrow \quad T(n) = 2F(n+1)-1 = \Theta(\varphi^n),\ \varphi = \tfrac{1+\sqrt{5}}{2} \approx 1{,}618$$

Слагаемое $+1$ — сам вызов, оба слагаемых $T(\cdot)$ — вызовы в поддеревьях. Например, $T(10) = 2F(11)-1 = 2 \cdot 89 - 1 = 177$. С мемоизацией каждая подзадача решается один раз, и число вызовов равно $2n - 1$ (для $n = 10$ — 19).

2.2. Задача Б: размен суммы

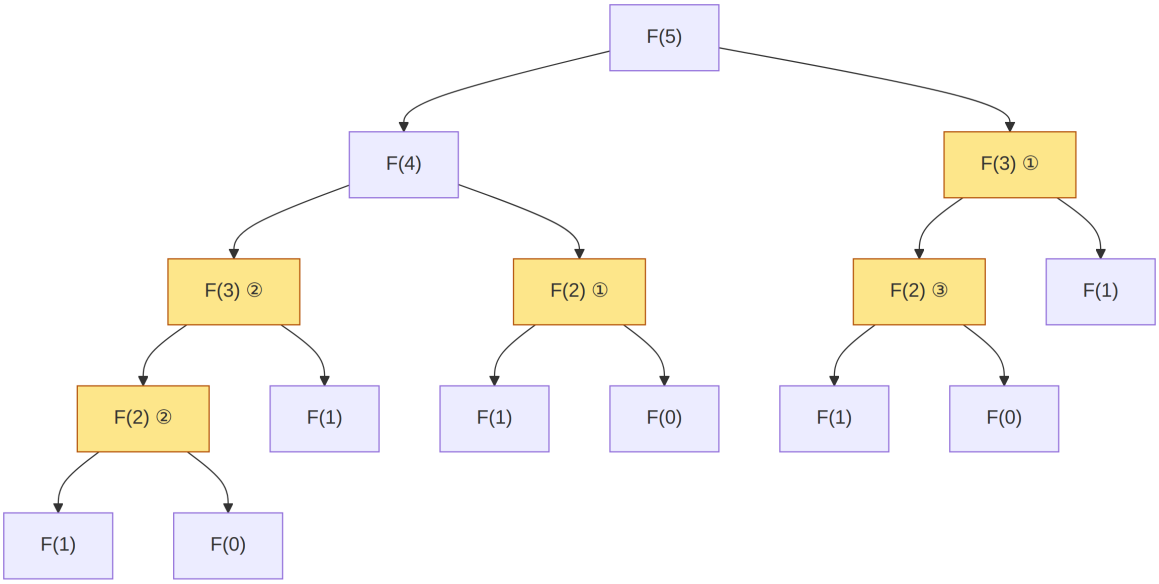
Дана система номиналов $c_1 = 1,\ c_2 = 5,\ c_3 = 12$ и сумма S. Пусть $D[s]$ — минимальное число монет, которым можно набрать сумму s. Тогда

$$D[0] = 0, \qquad D[s] = 1 + \min_{c_j \le s} D[s - c_j], \quad s = 1, \dots, S.$$

- $D[0] = 0$ — нулевая сумма набирается нулём монет;
- $\min_{c_j \le s}$ — перебор того, какая монета была выбрана **последней**; допустимы только номиналы, не превосходящие s;
- $D[s-c_j]$ — оптимальное решение оставшейся суммы (оптимальная подструктура: если набор для s оптимален, то набор для $s - c_j$ без последней монеты тоже оптимален, иначе его можно было бы улучшить);
- слагаемое 1 — сама выбранная монета.

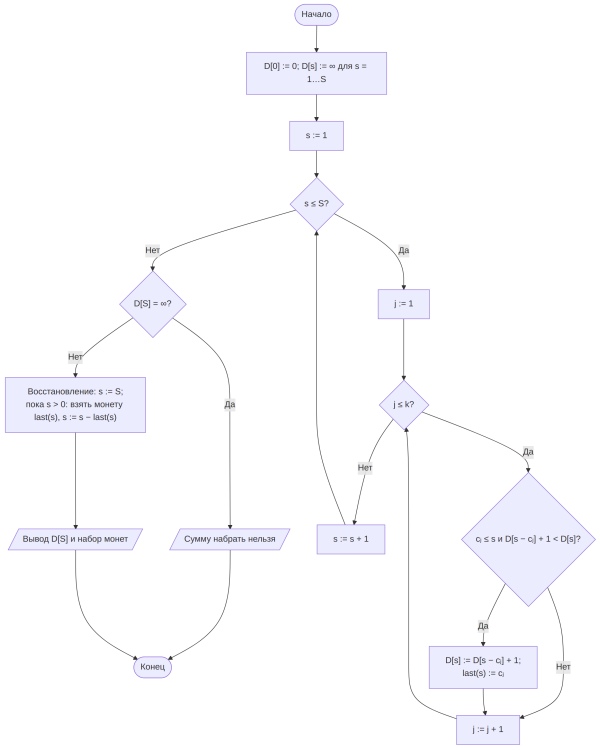
Трудоёмкость $\Theta(kS)$, где $k = 3$ — число номиналов; для $S = 180$ это 540 операций. Для восстановления набора в массив `last[s]` записывается монета, давшая минимум.

3. Дерево рекурсии для $F(5)$



Закрашены повторяющиеся подзадачи: $F(3)$ решается дважды, $F(2)$ — трижды, $F(1)$ — пять раз, $F(0)$ — три раза. Всего 15 вызовов ($= 2F(6) - 1$) при пяти различных подзадачах $F(0), \dots, F(5)$. С мемоизацией второе обращение к $F(3)$ и последующие обращения к $F(2)$ берутся из кэша, и дерево вырождается в цепочку.

4. Схема заполнения таблицы ДП (задача Б)



5. Листинг программы

Программа содержит три реализации чисел Фибоначчи со счётчиком вызовов, ДП для размена с восстановлением набора монет, наивную рекурсию для размена (демонстрация перекрывающихся подзадач), полный перебор для проверки, решение задачи о портфеле проектов и построение графиков.

```
"""
Лабораторная работа № 7. Вариант 14.
Задача А: числа Фибоначчи (наивная рекурсия, мемоизация, итерация), n = 10…32.
Задача Б: размен суммы S = 180, номиналы 1, 5, 12 (ДП, восстановление, сверка с перебором).
Дополнительное задание Г7: отбор проектов в портфель при ограниченном бюджете.
"""
import itertools
import json
import random
import sys
import time

sys.setrecursionlimit(10_000)

calls = 0                       # счётчик вызовов рекурсивной функции


# ---------------------------------------------- задача А: числа Фибоначчи
def fib_naive(n):
    """Наивная рекурсия: θ(φⁿ) вызовов."""
    global calls
    calls += 1
    if n < 2:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)


def fib_memo(n, cache=None):
    """Нисходящее ДП (мемоизация): 2n − 1 вызовов."""
    global calls
    calls += 1
    if cache is None:
        cache = {}
    if n < 2:
        return n
    if n in cache:
        return cache[n]
    cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    return cache[n]


def fib_iter(n):
    """Восходящее ДП: θ(n) времени, O(1) памяти."""
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a


def measure(func, n, loops):
    """Среднее время одного вызова (с) и число вызовов рекурсивной функции."""
    global calls
    calls = 0
    result = func(n)
    n_calls = calls
    t0 = time.perf_counter()
    for _ in range(loops):
        func(n)
    return (time.perf_counter() - t0) / loops, n_calls, result
```

```python
# ---------------------------------------------- задача Б: размен суммы
def change_dp(coins, S):
    """Минимальное число монет. Возвращает (число монет, список монет) или None.
    D[s] = 1 + min D[s − c]; в last[s] запоминается выбранная монета."""
    INF = float("inf")
    D = [0] + [INF] * S
    last = [0] * (S + 1)
    for s in range(1, S + 1):
        for c in coins:
            if c <= s and D[s - c] + 1 < D[s]:
                D[s] = D[s - c] + 1
                last[s] = c
    if D[S] == INF:
        return None
    chosen, s = [], S
    while s > 0:                        # восстановление набора монет
        chosen.append(last[s])
        s -= last[s]
    return D[S], sorted(chosen, reverse=True), D, last


def change_naive(coins, S):
    """Рекурсия без запоминания (для демонстрации перекрывающихся подзадач)."""
    global calls
    calls += 1
    if S == 0:
        return 0
    best = float("inf")
    for c in coins:
        if c <= S:
            best = min(best, 1 + change_naive(coins, S - c))
    return best


def change_brute(coins, S):
    """Полный перебор: числа монет каждого номинала (только для проверки)."""
    best = None
    for counts in itertools.product(*[range(S // c + 1) for c in coins]):
        if sum(k * c for k, c in zip(counts, coins)) == S:
            total = sum(counts)
            if best is None or total < best:
                best = total
    return best


def change_greedy(coins, S):
    """Жадный размен (для сопоставления): всегда наибольший номинал."""
    count = 0
    for c in sorted(coins, reverse=True):
        count += S // c
        S %= c
    return count


# ---------------------------------------- Г7: портфель проектов (рюкзак)
def portfolio_dp(costs, profits, budget):
    """Рюкзак 0/1: возвращает (прибыль, индексы проектов, строка D[N][·] по бюджетам)."""
    n = len(costs)
    D = [[0] * (budget + 1) for _ in range(n + 1)]
```

```
    for i in range(1, n + 1):
        c, p = costs[i - 1], profits[i - 1]
        for b in range(budget + 1):
            D[i][b] = D[i - 1][b]
            if c <= b and D[i - 1][b - c] + p > D[i][b]:
                D[i][b] = D[i - 1][b - c] + p
    return D


def portfolio_restore(D, costs, budget):
    chosen, b = [], budget
    for i in range(len(costs), 0, -1):
        if D[i][b] != D[i - 1][b]:
            chosen.append(i - 1)
            b -= costs[i - 1]
    return sorted(chosen)


def portfolio_brute_curve(costs, profits, budget):
    """Полный перебор подмножеств: лучшая прибыль для каждого бюджета 0…budget."""
    best_by_cost = [0] * (budget + 1)
    for mask in range(1 << len(costs)):
        c = p = 0
        for i in range(len(costs)):
            if mask >> i & 1:
                c += costs[i]
                p += profits[i]
        if c <= budget and p > best_by_cost[c]:
            best_by_cost[c] = p
    curve, cur = [], 0
    for b in range(budget + 1):
        cur = max(cur, best_by_cost[b])
        curve.append(cur)
    return curve


# ---------------------------------------------------------------- графики
def make_plots(res):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    fib = res["fib_ext"]
    ns = [r["n"] for r in fib]
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.3))
    axes[0].semilogy(ns, [r["calls_naive"] for r in fib], "o-", label="наивная рекурсия")
    axes[0].semilogy(ns, [r["calls_memo"] for r in fib], "s-", label="мемоизация")
    axes[0].set_title("Число вызовов функции")
    axes[1].semilogy(ns, [r["t_naive"] for r in fib], "o-", label="наивная рекурсия")
    axes[1].semilogy(ns, [r["t_memo"] for r in fib], "s-", label="мемоизация")
    axes[1].semilogy(ns, [r["t_iter"] for r in fib], "^-", label="итерация")
    axes[1].axhline(10, color="gray", ls="--", lw=1)
    axes[1].set_title("Время вычисления, с (пунктир — 10 с)")
    for ax in axes:
        ax.set_xlabel("n"); ax.grid(True, which="both", alpha=.3); ax.legend(fontsize=8)
    plt.tight_layout(); plt.savefig("img/lr7_fib.png", dpi=170); plt.close()

    pf = res["portfolio"]
    plt.figure(figsize=(7, 4.3))
    plt.step(range(len(pf["curve"])), pf["curve"], where="post")
```

```
    plt.axvline(pf["budget"], color="red", ls="--", lw=1, label=f"бюджет {pf['budget']} млн руб.")
    plt.xlabel("бюджет, млн руб."); plt.ylabel("максимальная прибыль портфеля, млн руб.")
    plt.title("Зависимость прибыли портфеля от бюджета"); plt.grid(True, alpha=.3); plt.legend()
    plt.tight_layout(); plt.savefig("img/lr7_portfolio.png", dpi=170); plt.close()


# ---------------------------------------------------------------------- main
if __name__ == "__main__":
    res = {}

    # --- Часть 1. Числа Фибоначчи, n = 10…32
    print(f"{'n':>4} {'наивная, с':>12} {'вызовов':>12} {'мемо, с':>10} {'вызовов':>9} {'итер.,
     ↪  с':>10} {'F(n)':>12}")
    res["fib"] = []
    for n in [10, 15, 20, 25, 30, 32]:
        loops_naive = 200 if n <= 20 else (5 if n == 25 else 1)
        t_n, c_n, r1 = measure(fib_naive, n, loops_naive)
        t_m, c_m, r2 = measure(fib_memo, n, 5000)
        t_i, _, r3 = measure(fib_iter, n, 20000)
        assert r1 == r2 == r3
        res["fib"].append({"n": n, "t_naive": t_n, "calls_naive": c_n, "t_memo": t_m,
                           "calls_memo": c_m, "t_iter": t_i, "F": r1})
        print(f"{n:>4} {t_n:>12.4f} {c_n:>12} {t_m:>10.6f} {c_m:>9} {t_i:>10.6f} {r1:>12}",
         ↪  flush=True)

    # --- предельное n для наивной рекурсии (ориентир — 10 с)
    res["fib_ext"] = [dict(r) for r in res["fib"]]
    n = 32
    while True:
        n += 1
        t_n, c_n, r1 = measure(fib_naive, n, 1)
        t_m, c_m, r2 = measure(fib_memo, n, 2000)
        t_i, _, r3 = measure(fib_iter, n, 5000)
        assert r1 == r2 == r3
        res["fib_ext"].append({"n": n, "t_naive": t_n, "calls_naive": c_n, "t_memo": t_m,
                               "calls_memo": c_m, "t_iter": t_i, "F": r1})
        print(f"{n:>4} {t_n:>12.3f} {c_n:>12}", flush=True)
        if t_n > 10:
            res["fib_limit"] = n
            break

    # --- Часть 2. Размен суммы S = 180, номиналы 1, 5, 12
    COINS, S = (1, 5, 12), 180
    count, chosen, D, last = change_dp(COINS, S)
    res["change"] = {"S": S, "coins": COINS, "count": count, "chosen": chosen,
                     "D_head": D[:31], "last_head": last[:31]}
    print(f"\nРазмен S = {S}: {count} монет, набор {chosen}")

    # сверка с полным перебором: все суммы 0…180 для системы варианта и случайные системы
    for s in range(S + 1):
        assert change_dp(COINS, s)[0] == change_brute(COINS, s), s
    rng = random.Random(14)
    for _ in range(100):
        coins = tuple(sorted({1, *rng.sample(range(2, 16), rng.randint(1, 3))}))
        s = rng.randint(1, 40)
        assert change_dp(coins, s)[0] == change_brute(coins, s), (coins, s)
    res["change"]["greedy_fail"] = [s for s in range(1, S + 1)
                                    if change_greedy(COINS, s) != change_dp(COINS, s)[0]]
    print("Проверка с перебором пройдена; жадность ошибается на",
     ↪  len(res["change"]["greedy_fail"]), "суммах")
```

```
# рекурсия без запоминания против ДП
res["change_naive"] = []
for s in (10, 20, 30, 40, 50, 60):
    calls = 0
    t0 = time.perf_counter(); v = change_naive(COINS, s); t = time.perf_counter() - t0
    res["change_naive"].append({"S": s, "calls": calls, "t": t, "dp_cells": s * len(COINS)})
    print(f"S = {s}: вызовов {calls}, {t:.4f} c", flush=True)
    if t > 10:
        break

# --- Г7. Портфель проектов: 12 проектов, затраты и прибыль в млн руб.
rng = random.Random(14)
N = 12
costs = [rng.randint(8, 45) for _ in range(N)]
profits = [rng.randint(12, 120) for _ in range(N)]
total = sum(costs)
D2 = portfolio_dp(costs, profits, total)
curve = D2[N][:]
assert curve == portfolio_brute_curve(costs, profits, total), "ДП и перебор расходятся"
B0 = 100
items = portfolio_restore(D2, costs, B0)
res["portfolio"] = {"costs": costs, "profits": profits, "total": total, "curve": curve,
                    "budget": B0, "items": items, "profit": curve[B0],
                    "cost": sum(costs[i] for i in items)}
print("Портфель при бюджете", B0, ":", items, curve[B0])

with open("res/lr7.json", "w", encoding="utf-8") as f:
    json.dump(res, f, ensure_ascii=False)
make_plots(res)
```

6. Таблицы измерений и графики

6.1. Условия измерений

Параметр	Значение
Операционная система	Linux 6.18 (x86_64), виртуальная машина
Процессор	Intel Xeon, 2,10 ГГц, 1 виртуальное ядро
Оперативная память	4 ГБ
Интерпретатор	CPython 3.12.3
Способ замера времени	`time.perf_counter()`, перед каждым прогоном вызывается `gc.collect()`

Время быстрых реализаций — среднее по многим повторам (мемоизация — 5000, итерация — 20 000), для наивной рекурсии при $n \geq 30$ — один прогон. Число вызовов подсчитано глобальным счётчиком.

6.2. Три реализации чисел Фибоначчи (задача А)

Таблица 2. Время и число вызовов, $n = 10 \dots 32$

n	Наивная, с	Вызовов	Мемоизация, с	Вызовов	Итеративная, с	$F(n)$
10	0,0000	177	0,000001	19	0,000000	55
15	0,0001	1 973	0,000002	29	0,000000	610
20	0,0011	21 891	0,000003	39	0,000000	6 765
25	0,0123	242 785	0,000004	49	0,000001	75 025
30	0,1348	2 692 537	0,000005	59	0,000001	832 040
32	0,3326	7 049 155	0,000005	63	0,000001	2 178 309

При переходе от $n = 30$ к $n = 32$ число вызовов наивной версии выросло в 2,62 раза, то есть примерно в $\varphi^2 = 2{,}618$ раза: подтверждение оценки $\Theta(\varphi^n)$. Число вызовов с мемоизацией точно равно $2n - 1$ (63 при $n = 32$). При $n = 32$ наивная версия выполняет 7 049 155 вызовов за 0,33 с, мемоизация — 63 вызова за 4,8 мкс, итеративная — 0,5 мкс.

6.3. Предельное значение n для наивной рекурсии

Для определения предельного n измерения продолжены до первого значения, при котором время превысило 10 с.

Таблица 3. Продолжение измерений для наивной рекурсии ($n \geq 32$)

n	Время наивной версии, с	Вызовов	Рост времени к предыдущему n
32	0,33	7 049 155	—
33	0,58	11 405 773	—
34	0,93	18 454 929	1,60
35	1,41	29 860 703	1,51
36	2,45	48 315 633	1,74
37	3,78	78 176 337	1,54
38	6,05	126 491 971	1,60
39	9,90	204 668 309	1,64
40	15,97	331 160 281	1,61

Время растёт в среднем в 1,61 раза на единицу n (регрессия $\ln T$ по n для $n \geq 30$), число вызовов — в 1,618 раза (регрессия для $n \geq 20$) — оба значения близки к $\varphi \approx 1{,}618$ (основание экспоненты определено по наклону прямой на логарифмической шкале). Время 10 с достигается между $n = 39$ (9,9 с) и $n = 40$ (16,0 с, 331 160 281 вызовов): **предельным значением следует считать** $n = 40$; реализации с мемоизацией и итерацией на этом n работают за микросекунды.

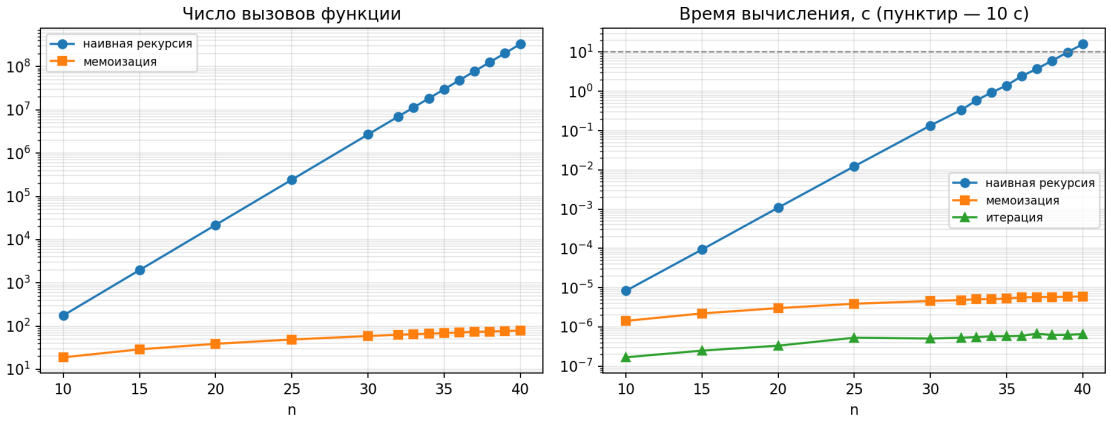


Рис. 1. Число вызовов и время вычисления для n до 40 (логарифмические шкалы по оси ординат)

6.4. Задача Б: размен суммы без запоминания и с ДП

Для той же задачи Б наивная рекурсия без запоминания также экспоненциальна. Это показывает, что условие перекрывающихся подзадач выполняется и здесь.

Таблица 4. Размен суммы с номиналами 1, 5, 12: рекурсия против ДП

S	Вызовов рекурсии без запоминания	Время, с	Ячеек таблицы ДП ($S \cdot k$)
10	33	0,0000	30
20	655	0,0001	60
30	12 613	0,0025	90
40	243 607	0,0575	120
50	4 702 949	1,0189	150
60	90 795 037	19,1863	180

При увеличении S на 10 число вызовов растёт примерно в 19,3 раза (около 1,34 раза на единицу S), тогда как таблица ДП содержит лишь kS ячеек: для $S = 60$ это 180 ячеек против почти $9{,}1 \cdot 10^7$ вызовов.

7. Оптимальное решение задачи Б

Для $S = 180$ и номиналов 1, 5, 12 динамическое программирование даёт **15 монет**: набор — **15 монет номиналом 12** ($15 \cdot 12 = 180$). Это единственный оптимальный набор: наибольший номинал равен 12, поэтому 14 монет дают не более $14 \cdot 12 = 168 < 180$, а 15 монет дают ровно 180 только в случае, когда все они имеют номинал 12 (замена любой монеты на меньшую уменьшает сумму).

Восстановление набора: $last[180] = 12$, затем $180 \to 168 \to 156 \to \ldots \to 12 \to 0$ — пятнадцать шагов с монетой 12.

Таблица 5. Начало таблицы ДП: минимальное число монет $D[s]$ и последняя выбранная монета

s	$D[s]$	монета	s	$D[s]$	монета	s	$D[s]$	монета
0	0	—	11	3	1	22	3	5
1	1	1	12	1	12	23	4	1
2	2	1	13	2	1	24	2	12
3	3	1	14	3	1	25	3	1
4	4	1	15	3	5	26	4	1
5	1	5	16	4	1	27	4	5
6	2	1	17	2	5	28	5	1
7	3	1	18	3	1	29	3	5
8	4	1	19	4	1	30	4	1
9	5	1	20	4	5			
10	2	5	21	5	1			

Сверка с полным перебором (перебор числа монет каждого номинала) выполнена для **всех** сумм $s = 0 \ldots 180$ и дополнительно для 100 случайных систем номиналов — расхождений нет (проверки `assert` в программе).

Замечание. Система 1, 5, 12 не является «канонической»: жадный алгоритм (всегда брать наибольший номинал) даёт неоптимальный результат на 56 из 180 сумм. Например, для $s = 15$ жадный алгоритм выдаёт $12 + 1 + 1 + 1$ (4 монеты), а оптимум — $5 + 5 + 5$ (3 монеты, строка $s = 15$ таблицы 5). Для суммы варианта $S = 180$ оба подхода совпадают, но это частный случай.

8. Дополнительное задание Г7: отбор проектов в портфель при ограниченном бюджете

8.1. Постановка

Компания рассматривает 12 инвестиционных проектов. Проект i требует затрат c_i (млн руб.) и приносит прибыль p_i (млн руб.); проект либо финансируется целиком, либо нет. Требуется выбрать набор проектов с максимальной суммарной прибылью при суммарных затратах не более бюджета B. Это задача о рюкзаке 0/1: вес — затраты, ценность — прибыль, вместимость — бюджет.

$$D[i][b] = \begin{cases} D[i-1][b], & c_i > b, \\ \max\bigl(D[i-1][b],\ D[i-1][b-c_i] + p_i\bigr), & c_i \le b, \end{cases} \qquad D[0][b] = 0.$$

Значение $D[12][b]$ — максимальная прибыль при бюджете b, поэтому одна заполненная таблица даёт весь график зависимости прибыли от бюджета. Исходные данные сгенерированы программой (фиксированное зерно): затраты 8…45 млн руб., прибыль 12…120 млн руб.

Таблица 6. Проекты

Проект	Затраты, млн руб.	Прибыль, млн руб.	Прибыль / затраты	В портфеле при бюджете 100
1	14	111	7,93	да
2	41	27	0,66	—
3	23	45	1,96	—
4	25	40	1,60	—
5	24	52	2,17	—
6	26	57	2,19	—
7	12	115	9,58	да
8	36	45	1,25	—
9	27	58	2,15	—
10	37	114	3,08	да
11	33	92	2,79	да
12	33	92	2,79	—

8.2. Решение для бюджета 100 млн руб.

Оптимальный портфель: проекты №1, 7, 10, 11; затраты 14 + 12 + 37 + 33 = 96 млн руб. (из 100), прибыль 111 + 115 + 114 + 92 = **432 млн руб.** Результат ДП сверен с полным перебором всех $2^{12} = 4096$ подмножеств **для всех бюджетов** $0 \dots 331$ — кривые совпали.

8.3. Зависимость прибыли от бюджета

Таблица 7. Максимальная прибыль портфеля при различных бюджетах

Бюджет B, млн руб.	Макс. прибыль, млн руб.	Прибыль на 1 млн бюджета
0	0	—
50	278	5,56
100	432	4,32
150	547	3,65
200	644	3,22
250	736	2,94
331	848	2,56

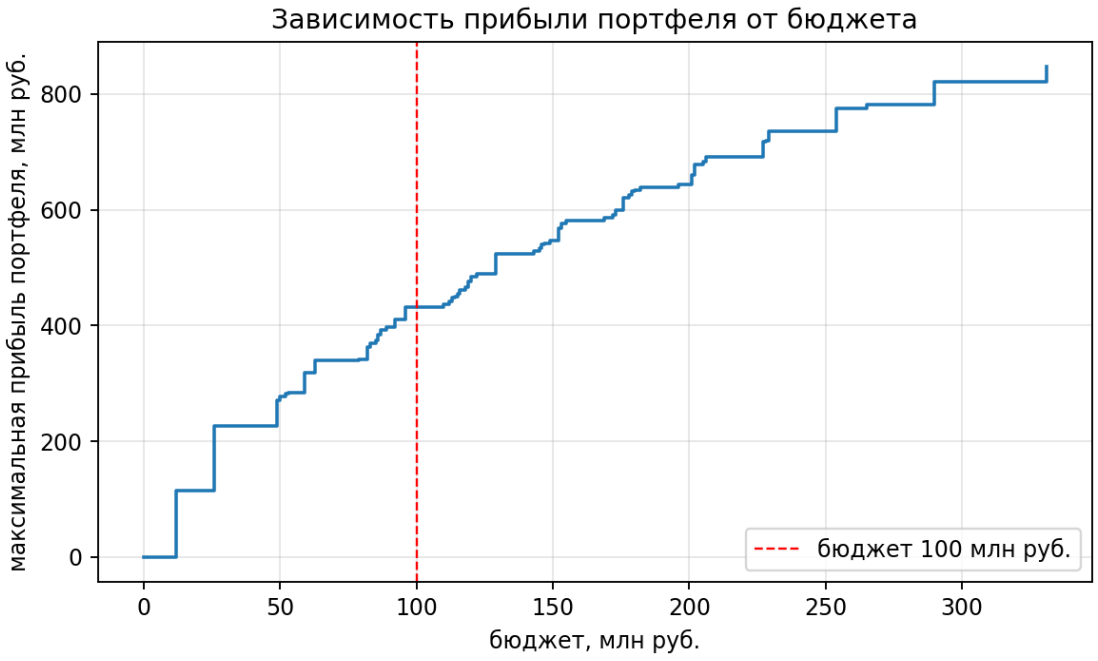


Рис. 2. Зависимость максимальной прибыли портфеля от бюджета

График — ступенчатая неубывающая функция: прибыль растёт скачками при бюджетах, позволяющих добавить очередной проект. Эффективность вложений убывает: первые 50 млн руб. приносят 5,56 млн прибыли на 1 млн бюджета, участок 100–150 млн — 2,30, участок 150–200 млн — 1,94. При бюджете 331 млн руб. и выше финансируются все проекты (прибыль 848 млн руб.), и дальнейшее увеличение бюджета прибыль не повышает. Такой график позволяет руководителю увидеть, при каком бюджете дополнительные вложения перестают окупаться.

9. Выводы

1. Наивная рекурсия для чисел Фибоначчи экспоненциальна: число вызовов равно $2F(n+1)-1$ и растёт в 1,618 раза на единицу n (теоретически в $\varphi \approx 1{,}618$). При $n = 32$ требуется свыше семи миллионов вызовов, а при $n = 40$ время превышает 10 с (16,0 с).
2. Мемоизация сокращает число вызовов до $2n-1$, то есть до линейного, ценой $\Theta(n)$ дополнительной памяти под кэш. Итеративная версия ещё быстрее (0,5 мкс против 4,8 мкс при $n = 32$) и требует $O(1)$ памяти, так как не тратит время на вызовы функций и хранение кэша.
3. Задача о размене тоже обладает перекрывающимися подзадачами: рекурсия без запоминания при $S = 60$ делает около $9 \cdot 10^7$ вызовов, ДП — 180 операций. Для $S = 180$ и номиналов 1, 5, 12 оптимум — 15 монет по 12.
4. Результат ДП совпал с полным перебором для всех сумм от 0 до 180 и на 100 случайных системах номиналов, а восстановление набора монет через массив `last` не требует повторных вычислений.
5. Для системы номиналов 1, 5, 12 жадная стратегия ошибается на 56 суммах из 180 (например, $s = 15$), поэтому правильность размена нельзя принимать за очевидную: ДП даёт точный ответ при любой системе номиналов.
6. Задача отбора проектов в портфель решается точно методом ДП за $\Theta(NB)$: при бюджете 100 млн руб. оптимальный портфель из проектов №1, 7, 10, 11 даёт 432 млн руб. прибыли. График зависимости прибыли от бюджета показывает убывающую отдачу и бюджет, после которого вложения не окупаются.

10. Ответы на контрольные вопросы

1. Что такое дерево рекурсии? Как по нему оценить трудоёмкость алгоритма? Это дерево, узлы которого — вызовы рекурсивной функции, а потомки узла — вызовы, порождённые им. Число узлов равно числу вызовов, поэтому трудоёмкость равна числу узлов, умноженному на стоимость обработки одного узла. Для чисел Фибоначчи дерево имеет $2F(n+1)-1 = \Theta(\varphi^n)$ узлов: для $n = 5$ — 15 узлов (раздел 3), для $n = 32$ — 7 049 155.

2. Почему наивное вычисление чисел Фибоначчи экспоненциально, хотя ответ вычисляется за $\Theta(n)$ операций итеративно? Потому что одни и те же подзадачи решаются многократно: $F(n-2)$ вычисляется и из $F(n)$, и из $F(n-1)$, и повторы множатся на каждом уровне (на рис. дерева $F(1)$ встречается 5 раз при $n = 5$). Различных подзадач всего $n+1$, но вызовов — $2F(n+1)-1$. Итеративная версия вычисляет каждую подзадачу один раз (32 сложения при $n = 32$ против 7 049 155 вызовов).

3. Сформулируйте два условия применимости динамического программирования. Оптимальная подструктура: оптимальное решение содержит оптимальные решения подзадач (для размена — набор для суммы s без последней монеты оптимален для $s - c$). Перекрывающиеся подзадачи: число различных подзадач невелико (полиномиально, для размена — S), но каждая встречается многократно (таблица 4).

4. Чем динамическое программирование отличается от парадигмы «разделяй и властвуй»? При «разделяй и властвуй» подзадачи независимы и не повторяются (сортировка слиянием), поэтому запоминание бесполезно. В ДП подзадачи перекрываются, и ключевой приём — сохранение решённых подзадач, что превращает экспоненциальную рекурсию в полиномиальный алгоритм.

5. В чём различие нисходящей и восходящей форм? Каковы преимущества каждой? Нисходящая форма (мемоизация) сохраняет естественную рекурсивную запись и решает только нужные подзадачи, но несёт накладные расходы на вызовы и ограничена

глубиной рекурсии. Восходящая заполняет таблицу в порядке возрастания размера подзадач, без рекурсии, быстрее и допускает экономию памяти, но требует продумать порядок заполнения. В опыте при $n = 32$: мемоизация — 4,8 мкс и 63 вызова, итерация — 0,5 мкс.

6. Запишите рекуррентное соотношение задачи о рюкзаке и поясните смысл каждой ветви. $D[i][w] = D[i-1][w]$, если $w_i > w$ (предмет i не помещается — решение такое же, как для первых $i-1$ предметов); $D[i][w] = \max(D[i-1][w],\ D[i-1][w-w_i] + v_i)$, если $w_i \le w$ (выбор между «не брать» i-й предмет и «взять» его: тогда остаётся вместимость $w - w_i$, и добавляется ценность v_i). Ответ — $D[N][W]$. В работе то же соотношение использовано в задаче Г7 (раздел 8.1).

7. Почему сложность $\Theta(NW)$ называется псевдополиномиальной? Она полиномиальна по числовому значению W, но размер входа — длина записи числа, $\log_2 W$ бит, поэтому относительно размера входа время экспоненциально ($W = 2^{\log_2 W}$). Например, бюджет 100 млн руб. даёт таблицу из $12 \cdot 100$ ячеек, а тот же бюджет в копейках (10^{10}) сделал бы таблицу непригодной по памяти. Поэтому задача остаётся NP-трудной, и метод эффективен лишь при умеренных W.

8. Как восстановить состав оптимального решения по таблице ДП? Обратным проходом: для рюкзака — начиная с $D[N][W]$, если $D[i][w] \neq D[i-1][w]$, предмет i включён, и далее рассматривается $D[i-1][w-w_i]$. Для размена — по массиву `last[s]`: берётся монета `last[s]`, $s := s - last[s]$, пока $s > 0$ (для $S = 180$ получено 15 раз по 12). В задаче Г7 обратный проход дал проекты №1, 7, 10, 11.

9. Как сократить память в задаче о рюкзаке до $\Theta(W)$? Что при этом теряется? Строка $D[i]$ зависит только от $D[i-1]$, поэтому достаточно одного массива длиной $W+1$, обновляемого по w в убывающем порядке (чтобы предмет не использовался дважды). Теряется возможность обратного прохода по таблице: сам набор предметов восстановить нельзя, остаётся только значение; для восстановления нужно дополнительно хранить выбор (биты $N \times W$) или пересчитывать (схема Хиршберга).

10. Приведите пример задачи управления, сводящейся к задаче о рюкзаке. Отбор проектов в портфель при ограниченном бюджете (решено в разделе 8): затраты проекта — вес, прибыль — ценность, бюджет — вместимость. Другие примеры: загрузка транспорта по грузоподъёмности, выбор рекламных кампаний при ограниченном бюджете, формирование ассортимента при ограниченной площади склада.